π Tantum

a.k.a. Tempus Tantum

school timetable generation system

"Omnia, Lucili, aliena sunt, tempus tantum nostrum est."
Lucius Annaeus Seneca, *Epistulae morales ad Lucilium*, I, 1

Manual

A method for Italian-school timetabling

(English edition)

From an idea by Fernando Gargiulo, Giovanni Borghi,
Matteo Mariani, Stefano Bertozzi

Giovanni Borghi

May 7, 2026

Repository: https://github.com/gborghi/timetable
Documentation source: docs/

Colophon

This English edition is set in EB Garamond for body text and Lato for headings, with Inconsolata for code blocks. Compilation is done with lualatex (TeX Live), biber for bibliography and makeindex for the analytical index. The LaTeX source lives in docs/manual_en.tex with chapters in docs/manual/chapters_en/; the bibliography is shared with the Italian edition in docs/manual/bibliography.bib. The build pipeline is in docs/build_manual.sh which emits **both** manual.pdf (Italian) and manual_en.pdf (English) in a single invocation.

Note on translation: the Italian edition remains the primary reference. This English edition fully translates the introduction, the BP-related chapters and the benchmark chapter; all other chapters carry an English summary that points the reader to the corresponding Italian chapter for the full narrative. Where Italian-school context is necessary (for instance the legal frame of the *contratto collettivo nazionale*, the role of the *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added in the English text.

The repository version corresponds to the most recent commit visible in git log –oneline – docs/. Errors and suggestions can be reported as issues on the GitHub repository at https://github.com/gborghi/timetable.

Contents

List of Figures 6

List of Tables 7

Introduction 8

1 The timetabling problem 9

2 piTantum overview 10

3 The timetable as a structural object 11

4 Installation 12

5 Constraints (HARD + SOFT) 13
5.1 Plessi: multiple sites in the same school . . . 13
5.2 Canonical patterns and seeded legacy HARDs (Step 3b-3e) . . . 15

6 Optimization workflow 17

7 UI guide 19

8 Launcher 20

9 CP-SAT method 21

10 Spectral decomposition 22

11 Metaheuristics 23

12 Hall pre-check 24

13 Lagrangian relaxation and Column Generation 25

14 Monte Carlo sensitivity 26

15 Graph-based diagnostics 27

16 Statistical diagnostics 28

17 Constraint DSL method 29

18 Architecture 30
18.1 Stack . . . 30
18.2 Three-tier overview . . . 30
18.3 Solver architecture: from DSL to CP-SAT . . . 30
18.4 Backend lifecycle . . . 32

19 Data model 33

20 Generic constraint DSL 34
20.1 Philosophy . . . 34
20.2 BNF grammar . . . 34

20.3 DSL → CP-SAT compilation (Step 1) . . . 35
20.4 Slot predicates: consecutive, same_day . . . 35
20.5 The l.classroom.plesso path . . . 35
20.6 Canonical pattern vocabulary (Step 3b) . . . 35
20.7 Plesso commuting via DSL (Step 3c) . . . 36
20.8 Seed legacy HARDs via DSL (Step 3d-3e) . . . 36
20.9 SOFT constraints with weight . . . 36
20.10 Scope: where to save the constraint . . . 37
20.11 Built-in predicates: consecutive_days, same_day, consecutive . . . 38
20.12 Arithmetic in the DSL . . . 38
20.13 Real-world cases (Rossi and friends) . . . 38
20.14 The four compilation families . . . 40
20.15 Logic DNF vs forall/count . . . 40
20.16 Fourteen canonical pragmas (Phase A and Phase B) . . . 40
20.17 Workflow: from the Teacher tab to the solver . . . 41
20.18 Three narrated cases . . . 42
20.19 DSL architecture diagram . . . 43

21 Advanced optimization techniques 45
21.1 Hall pre-check . . . 45
21.2 Column Generation and advanced Branch-and-Price . . . 45
21.3 Branch-and-Price MVP scaffold (Step 4) . . . 46
21.4 Lagrangian relaxation . . . 46
21.5 Phase A: the formal *day count* model . . . 46
21.6 Phase B: four decomposition algorithms compared . . . 47
21.7 cp_sat_scope=week: the monolithic model . . . 47
21.8 Branch-and-Price: anatomy of one iteration . . . 48
21.9 Metaheuristics: a decision matrix . . . 51

22 Statistical diagnostics tab 55

23 REST API 56

24 Extending the system 57

25 Repository on GitHub 58

26 Benchmarks and results 59
26.1 The six profiles . . . 59
26.2 End-to-end pipeline times . . . 59
26.3 Branch-and-Price on HUGE and MEGA . . . 60
26.4 When Branch-and-Price is worth it . . . 62

27 Lessons learned 63

28 Glossary (narrative form) 64

29 Reference 65

Bibliography 66

Indice analitico 67

List of Figures

20.1 The 14 canonical pragmas grouped by pipeline layer. Phase A pragmas are the building blocks of DayCountModel; Phase B pragmas feed PhaseBDaySolver; the last two forms (class_subject_same_day and teacher_class_consecutive_days) are expressed directly via general_dsl without a dedicated helper. 41

20.2 DSL pipeline: from the modal in the UI to the four compilation back-ends. The *validate* button only fires the parser (returning errors live); *save* runs the full loop: persist to constraints_general, reload at the next POST /api/optimize, evaluate post-hoc. . . 42

20.3 Architecture of the general DSL: the same grammar and AST, four translation back-ends. Evaluating a solution (post-hoc evaluator) and building a model (compiler to CP-SAT) are two different traversals of the same tree. 44

21.1 Phase B wall-clock per decomposition method, four scales. Monolithic times out on superhuge; the four decompositions converge in comparable time, with a slight edge for spectral and metis on the larger regimes. Log scale to fit the dispersion small→superhuge. 49

21.2 Five composable scalability techniques around the *Branch-and-Price* loop: Ryan-Foster recursive tree, box-step dual stabilization, column management, parallel pricing, pricing-in-nodes (Step 4). All five are on by default in mode="branch-and-price". 50

21.3 Master LP soft-cost convergence per granularity (small synthetic profile). Finer granularities (curriculum, teacher-class-subject) reach soft cost 0; coarser ones plateau because patterns cannot change enough. Iter=1 no_improving_column runs correspond to seed catalogues already optimal. 52

21.4 Pricing time per granularity, with dc_source between Phase A (rigid) and weekly (cp_sat_scope=week). Small scenario shows no practical difference. 53

21.5 MEGA real (100 classes, 178 teachers, 2653 cattedra-day): breakdown of total wall-clock 1493 s into Phase A (301 s) + Branch-and-Price (1192 s) + post (<0.1 s). 53

21.6 Total time vs scenario tightness for the three dominant combinations (Phase B alone, + SA, + ALNS). The ivory band is the metaheuristic spread, growing with tightness. 54

26.1 Branch-and-Price convergence time (log scale). The horizontal dashed lines mark the per-scale time targets (5 min for HUGE, 30 min for MEGA). 60

26.2 Final column-pool size after the Branch-and-Price loop. 61

List of Tables

20.1 Four compilation families, one grammar. They were historically four separate sub-DSLs; from v0.4 the parser is unified (general_dsl.py) and produces a shared AST from which each family extracts what it needs. 40

20.2 The logical DNF is a subset of the general grammar. Migrating from DNF to general form is always possible (helper logical_unavailability_to_dsl); the reverse usually requires an abstraction the constraint does not have. 40

21.1 The five Phase A pragmas. All emit CP-SAT bytes identical to the legacy solve_phase_a (zero-drift); migration is tracked in webui/backend/tests/test_dsl_intvar_pragmas.py. 47

21.2 Phase B decomposition: pros and cons. *spectral* is the default; *temporal* is preferred when daily constraints dominate (last-minute substitutions); *metis* on MEGA when K-means randomness yields imbalanced clusters. 48

21.3 Nine pricer granularities. Each defines what a pattern *is* and therefore the CP-SAT model the pricer builds. None dominates the others. 51

21.4 Seven metaheuristics, all selectable from the Workflow tab via the "Post-Phase B" dropdown. The default cascade is LNS -> ALNS -> SA. 51

26.1 End-to-end pipeline times per profile. Phase A includes teacher-class assignment and CP-SAT matching. Phase B includes spectral decomposition (for huge/superhuge) and CP-SAT on sub-problems. 59

26.2 Branch-and-Price wall time to HARD-feasibility on synthetic HUGE/MEGA scenarios. Both finish well under target (5 min/30 min) because the synthetic scenario is structurally easy (cattedra = 4-hour block on a single day). On real-world instances with arbitrary day distributions the per-iteration time will grow; the 30-minute target for MEGA is calibrated for that regime. 60

26.3 Branch-and-Price on real MEGA before and after the DW seeding fix. Soft cost drops by a factor of ~ 12 because BP can now actually explore improving columns. Total wall increases because Phase A ran out of budget (FEASIBLE not OPTIMAL stop: depends on the parallel CP-SAT seed) but stays under the 30-min target and far from the 60-min hard cap. Source: tests/benchmarks/results/mega_bp_real_v2.json. 61

26.4 Three runs on the same real MEGA. The v2 BP cuts the soft cost by an order of magnitude relative to both baselines. 62

Introduction

> "Omnia, Lucili, aliena sunt, tempus tantum nostrum est."
>
> Lucius Annaeus Seneca, *Epistulae morales ad Lucilium*, I, 1

πTantum is a web application that assists in the composition of Italian-school timetables. It originates from ideas discussed and developed by Fernando Gargiulo, Giovanni Borghi, Matteo Mariani and Stefano Bertozzi.

The school timetabling problem consists of assigning, to each *cattedra* – the pair formed by a teacher and a class for a specific subject – a set of weekly hours distributed over days and time slots, such that all the following constraints are honoured: teacher availability, curricular hour coverage, classroom and equipment compatibility, and the constraints that derive from the Italian national collective contract for the school sector. This is a combinatorial problem classified as NP-hard, which means that as a school grows in size there is no fast algorithm capable of finding an exact optimal solution.

πTantum delegates the actual solving to the CP-SAT solver in Google OR-Tools, chosen for its maturity in constraint programming applied to scheduling problems. To scale beyond the size that the solver would handle directly, the system decomposes the problem into smaller sub-problems via four alternative strategies: spectral decomposition of the bipartite class-teacher graph, temporal decomposition by day of the week, curriculum-based decomposition, and balanced k-way METIS partitioning. On top of these, a cascade of metaheuristics (Large Neighborhood Search, Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search and Variable Neighborhood Search) improves the quality of the solution found.

The application backend is written in Python on FastAPI; data persistence is delegated to SQLite. The user interface is based on SvelteKit (in Italian for the production deployment), organised into sixteen tabs that cover the whole lifecycle of a timetable: registry import, constraint configuration, generation, manual editing with real-time feasibility checks, and day-to-day handling of absences and substitutions.

The following chapters describe the problem formally, the entities involved, the user interface, and each of the algorithms that πTantum employs to compute the timetable. The exposition is intentionally multi-layered: a non-technical reader can follow the narrative thread without engaging the mathematical formulas, while a developer will find the implementation details, code references, and source-file pointers necessary to extend the system.

This English edition is a parallel translation of the Italian manual (`docs/manual.tex`). The Italian edition remains the primary reference; this English edition is intended for international readers and contributors. Where Italian-school context is necessary (e.g. the legal frame of the *contratto collettivo nazionale*, the role of *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added.

1 The timetabling problem

Formal statement of the school timetabling problem: cattedre as (teacher, class, subject) triples; weekly hour distribution across days and time slots; HARD constraints (no class/teacher overlap, hour coverage, consecutive hours starting at 8 with presence at h=11) and SOFT objectives (daily uniformity, free-day preferences). NP-hardness and the rationale for decomposition + metaheuristics.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/problema_timetabling.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

2 piTantum overview

Three-tier architecture: engine (CP-SAT solver core), backend (FastAPI + SQLite), frontend (SvelteKit). Pipelines exposed as async runs via /api/optimize/*. The 16 UI tabs covering registry, constraints, generation, manual editing, daily operations.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/panoramica_pitantum.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

3 The timetable as a structural object

The Lesson dict (teacher, class, subject, day, hour): 1 as the canonical representation; round-trip with the Solution table; the role of dc_value in encoding Phase A's day-count distribution.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/orario_oggetto_strutturale.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

4 Installation

Python 3.13 + ortools + scipy + matplotlib + FastAPI + SQLAlchemy + Alembic; SvelteKit for the frontend. Cypress + Playwright for E2E. docker-compose.test.yml orchestrates the test stack.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/installazione.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

5 Constraints (HARD + SOFT)

HARD: no overlap (teacher in one class at a time, class with one teacher per slot), hour coverage of all cattedre, daily start at h=8, consecutive hours, presence at h=11 (the H1/H2/H3 trio). Native locks. C1 (compresenza/coteach), C2 (sostegno DVA), C3 (potenziamento, parallel intra-class, group inter-class). SOFT: daily uniformity, free-day preferences, sixth-hour penalties, five/one day load.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/vincoli.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

5.1 Plessi: multiple sites in the same school

In Italy many secondary schools are spread across multiple physical sites – the *Sede Centrale* (main building), one or more *succursali* (branch buildings, e.g. *Succursale Via Garibaldi*), or a dedicated *plesso* for a specific track (e.g. *Plesso ITT* for the technical track). Moving teachers or classes between sites during the day costs real time and introduces constraints that cannot be modelled with H1/H2/H3 alone.

The **Plessi** module (tab /plessi) handles:

- **site registry** (name, code, address);
- **classroom** → **plesso** association (each classroom belongs to exactly one site);
- two kinds of behavioural rules: **commuting rules** (minimum gap to move between two sites) and **entity policies** (global constraints on how many sites a teacher or class may attend).

5.1.1 Commuting rules: the gap between two sites

A commuting rule applies to an ordered pair (from_plesso, to_plesso) and specifies the minimum number of hours that must separate two consecutive lessons of the same entity when the classroom changes site. Relevant fields:

- from_plesso_id, to_plesso_id: the site pair.
- entity_kind: who is affected (teacher, class, group).
- entity_id: optional – if NULL the rule is *kind-wide* (applies to every teacher / class / group); if set it applies only to that specific entity.
- min_gap_hours: minimum number of empty hours required between the last lesson at site A and the first lesson at site B.
- symmetric: if true the rule applies in the opposite direction (B→A) as well.
- priority: tie-breaker when several rules overlap.

Typical Italian example: between *Sede Centrale* and *Succursale Via Garibaldi* a 60-minute travel time is required. The matching rule is (Centrale, Garibaldi, kind=teacher, entity_id=NULL, min_gap_hours=1, symmetric=true): no teacher may have a lesson at Garibaldi at h=9 if they have one at Centrale at h=8.

Rule resolution always applies *specific beats generic*: if a rule with entity_id=42 exists for teacher Rossi, it overrides the kind-wide rule. Among rules of equal specificity, higher priority wins.

5.1.2 Entity policies: per-person global constraints

An entity policy is a *global* constraint (not tied to a specific site pair) on the behaviour of a teacher or class. Three policies are supported:

- any – no global constraint, only commuting rules apply. This is the default.
- single_plesso_per_day – on any given day the entity may have lessons in at most one site. Stricter than commuting rules: even with a 1-hour gap, switching sites mid-day is forbidden.
- single_plesso_total – the entity attends *only* site plesso_id for the entire week. Every classroom assigned to that entity must belong to that site.

Concrete Italian examples:

- *Teacher Rossi does not want mid-day site switches*: the office creates a policy (entity_kind=teacher, entity_id=Rossi, policy=single_plesso_per_day). On Monday all of Rossi's hours are at Centrale; on Tuesday all at Garibaldi; never a mixed day.
- *Class 3A (technical track) attends only Plesso ITT*: policy (entity_kind=class, entity_id=3A, policy=single_plesso_total, plesso_id=ITT). Every classroom that hosts 3A lessons must belong to Plesso ITT.
- *All teachers, by default, avoid mid-day site switches*: policy (entity_kind=teacher, entity_id=NULL, policy=single_plesso_per_day) – applies to every teacher except those with a more specific override.

Entity policies follow the same *specific beats generic* rule (per-entity override), then priority.

5.1.3 Pre-run validation

Before launching the optimisation the system runs a battery of checks on the plessi data (validate_plessi_rules). If anything is inconsistent the run is rejected with a list of violations. What is checked:

- empty or duplicate plesso code;
- commuting rule referencing a non-existent plesso;
- commuting rule with negative min_gap_hours or larger than the day length;
- duplicate kind-wide commuting rule (same site pair, same kind, NULL entity_id, defined twice);
- single_plesso_total entity policy with no plesso_id or with a non-existent site;
- entity policy with an unknown policy value;
- override for a non-existent entity (deleted teacher or class).

The GET /api/plessi/validate endpoint lets the UI display a live "plessi config OK / N issues" badge.

5.1.4 CP-SAT implementation

At solver level, commuting rules become *pairwise* constraints on the classroom-assignment variables:

$$\forall (h_a, h_b) \text{ adjacent such that } \text{room}(t,d,h_a) \in P_A,\ \text{room}(t,d,h_b) \in P_B:\ h_b - h_a \geq \text{min\_gap}_{P_A \to P_B}.$$

Entity policies become counter constraints: $\texttt{single\_plesso\_per\_day}$ is $\sum_P \mathbf{1}[\exists h : \text{room}(t,d,h) \in P] \leq 1$ for each day d; $\texttt{single\_plesso\_total}$ fixes the domain of the classroom variables to the target site only.

All such constraints are *hard* by default. The roadmap envisages a *soft* variant with penalty for sporadic emergency overrides (e.g. last-minute substitute teaching), but the current version treats every plesso constraint as HARD.

5.1.5 Plessi via DSL (Step 3a-3c)

Starting in Step 3a, PlessoCommutingRule rows are fully expressible as DSL clauses thanks to two language additions (cf. cap. 20):

- the $\texttt{l.classroom.plesso}$ chained path that, given a lesson, resolves the assigned room and then the plesso the room belongs to;
- the $\texttt{consecutive(l1.slot, l2.slot)}$ predicate, true when the two lessons fall on the same day and differ by one hour.

The legacy rule "Centrale → Garibaldi minimum 1-hour gap, teacher Rossi" translates to:

```
forall l1 in lessons where
        l1.classroom.plesso == 1
    and l1.teacher == "Rossi":
     forall l2 in lessons where
             l2.classroom.plesso == 2
         and l2.teacher == "Rossi"
         and consecutive(l1.slot, l2.slot):
          false
```

The helper $\texttt{plesso\_commuting\_rule\_to\_dsl(rule)}$ in $\texttt{engine/dsl\_translator.py}$ produces these clauses automatically from the ORM row. A regression test enforces zero-drift: the DSL path emits exactly the same forbidden slot pairs as the legacy $\texttt{plessi\_constraints.py}$ pipeline.

5.2 Canonical patterns and seeded legacy HARDs (Step 3b-3e)

Step 3b-3e introduces a vocabulary of canonical patterns recognised directly by the DSL → CP-SAT compiler. Each maps to a legacy HARD constraint that previously lived only as hardcoded encoding. They can now be authored in DSL and are also generated automatically by $\texttt{seed\_implicit\_hardcoded(profs)}$.

Accepted pragmas:

- $\texttt{no\_holes\_class("3B")}$ – no gaps in 3B's weekly schedule (legacy $\texttt{add\_class\_no\_holes}$).
- $\texttt{class\_present\_at\_hour("3B", 11)}$ – if 3B has any lesson on a day, the slot at hour 11 is busy (HARD-3, "no exit before noon").
- $\texttt{class\_day\_load\_in("3B", 0, 4, 5, 6)}$ – 3B's daily load is 0 (free day) or in [4, 6] hours (HARD-1 + HARD-2).

- teacher_max_per_day("Rossi", 5) – teacher Rossi has at most 5 hours on any day.
- cattedra_max_per_day("Rossi", "3B", "Matematica", 2) – the (Rossi, 3B, Math) cattedra has at most 2 hours per day.
- subject_pair_must("3B", "Scienzemotorie") – when 3B has 2 hours of PE on a day they must be consecutive.
- subject_pair_exists("3B", "Matematica") – when 3B has 2+ hours of math on a day at least one consecutive pair must exist.

Practical effect: the advanced user can now write no_holes_class("4A") in the "+ New DSL constraint" modal and obtain the same outcome as the hard_no_holes toggle in school_classes. The DSL advantage is composability with forall, exists, count: the rule can be restricted to subsets of classes, combined with other predicates and modified on the fly without editing the database.

6 Optimization workflow

The pipeline orchestrator: Hall pre-check, Phase A (assignment + day-count), Phase B (per-day slot placement), optional metaheuristic post-processing (LNS / ALNS / SA / TS / ILS / VNS), classroom assignment.

CP-SAT scope and Phase A mode (Phase 3)

Card 3 of /optimize now exposes two dropdowns that control how Phase B drives the CP-SAT solver:

- **CP-SAT scope** (cp_sat_scope, default day):
  - day: solve one weekday at a time, using Phase A's pre-computed weekly distribution (*day_count*) as a hard equality. Fast, scales to large schools, default.
  - week: a single MonolithicSolver(scope=None) call covers the whole week. More robust on schools with many cross-day constraints (co-teaching, support, group rotations) but significantly slower because the solver explores a six-times-larger search space.
- **Phase A mode** (phase_a_mode, default always):
  - always: Phase A always runs and day_count is enforced as a hard equality. Mandatory for cp_sat_scope=day.
  - skip: skip Phase A entirely; the week solver picks its own day distribution. Only valid with cp_sat_scope=week.
  - soft_hint: run Phase A but inject day_count as model.AddHint on the week solver – a warm start, not a hard constraint. Only valid with cp_sat_scope=week.

Cross-field rule (validated on both client and server): day requires always; week forbids always. The UI disables invalid options and auto-corrects phase_a_mode when cp_sat_scope changes.

Picking the right combination.

- Standard school, speed-first: day + always (the default).
- Small school with many co-teaching / support constraints: week + skip – Phase A's hard day_count can make a feasible schedule look infeasible.
- Large school with non-trivial cross-day constraints: week + soft_hint – keeps the warm start while letting the solver deviate.

Branch-and-Price V1 vs cp_sat_scope=week**.** An open question during Phase 3 was whether running BP V1 on top of a "loose" dc_value (Phase A skipped or used as a soft hint) would unlock more master iterations than the single no_improving_column pass observed in legacy day-mode – the intuition being that a non-binding cover demand should produce non-zero λ duals and improving columns.

The empirical answer, measured by tests/benchmarks/run_bp_v1_scope_week.py on a small (10 teachers / 3 classes) and a medium (4 teachers / 3 classes, dense cattedre) scenario, is *no*. All five V1 granularities (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject) always terminate with iters=1, cols=0, terminated=no_improving_column – both when dc_value comes from Phase A and when it is derived from the week solver's actual placement, even though the two distributions differ structurally on the medium scenario.

The reason is structural, not a Phase 3 regression: column_generation._seed_patterns produces greedy-optimal patterns relative to whatever dc_value you feed it. The V1 master picks the highest-weight seed and the pricing sub-problem reconstructs the same grid – reduced cost is non-negative because the per-teacher problem already saturates the duals. Changing the dc_value source changes the per-day distribution, not the fact that each teacher has a single "shape" that fits its demand greedily.

Practically: cp_sat_scope=week relaxes Phase B's day-distribution constraint, but it does *not* unlock more BP V1 iterations. For column-generation quality on larger schools the current lever is the V2 master (class, class-day, day, curriculum) or the iterative-diversified mode, not the week scope.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/workflow_di_ottimizzazione.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

7 UI guide

Tour of the SvelteKit tabs: dashboard, anagrafica (teachers / classes / students / classrooms / *ore*), assignments, monitor, optimize (with iterative-diversified and branch-and-price modes), diagnostics, runs.

Working hours configuration (Tab Ore)

The /ore tab lets the school administrator configure the working week:

- which days are active (default: Mon-Sat);
- how many timetable slots each day has and at what times (default: 6 slots, 08:00-14:00).

Slots are stored as 0-based indices internally (WorkingHourSlot.slot_index), with explicit start_time/end_time (HH:MM strings) used only for display in the weekly calendar view and reports. Days carry a position that the engine consumes as day_idx, plus a legacy_day_number (1..7) that keeps backward compatibility with the existing teacher_unavailability / class_unavailability / classroom_unavailability rows keyed on the historical DAYS=[1..6] / HOURS=[8..13] convention.

The engine reads this configuration through engine/working_hours_config.py, which falls back to the legacy hardcoded defaults whenever the working_days table is missing or empty so old DBs keep working unchanged.

The unavailability matrices on the Teachers / Classes / Classrooms tabs were migrated to a new reusable <WeeklyCalendarView /> component that consumes the same configuration: identical color palette, click-cycle and keyboard shortcuts (H/P/E/D/A/ N + click for HARD/PREFERRED/ENFORCED/-SOFT/ALLOWED/RESET) as the previous AvailabilityMatrix, but with columns driven by the active working days and slot times rendered from the configuration.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/guida_ui.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

8 Launcher

Single-binary launcher script that bootstraps the venv, runs the alembic migrations, starts uvicorn + the SvelteKit dev server.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/launcher.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

9 CP-SAT method

Model formulation in OR-tools cp_model: BoolVar slot[(t, cl, s, d, h)], hour coverage equalities, no-overlap constraints, soft penalties as objective coefficients. Integer scaling for the fractional duals in BP.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_cpsat.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

10 Spectral decomposition

Bipartite class-teacher graph; normalised Laplacian eigendecomposition; spectral clustering into K balanced clusters; per-cluster CP-SAT + ricucitura with the bridge teachers.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_spettrale.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

11 Metaheuristics

LNS (random_window, day_cluster, worst_fit_day, teacher_day, classroom_day, single_class_week destroy ops + cp_sat_window repair), Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search, Variable Neighborhood Search.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_metaeuristiche.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

12 Hall pre-check

Hall's marriage theorem applied to the bipartite (class, subject) -> qualified-teachers graph as a fast pre-check (1-100 ms) before launching Phase A.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_hall.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

13 Lagrangian relaxation and Column Generation

Dualisation of the bridge constraints between clusters; subgradient ascent on the Lagrangian multipliers; SA-based per-iteration refinement. Column Generation: master LP + per-teacher CP-SAT pricing; Branch-and-Price extension with all 9 granularities (see ch. **??**).

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_lagrangian.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

14 Monte Carlo sensitivity

Random subset sampling to estimate the Hall violation probability under perturbations; useful as a pre-check robustness indicator.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_montecarlo.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

15 Graph-based diagnostics

Bipartite class-teacher graph: density, modularity (Newman-Girvan), betweenness centrality. Identifies bridge teachers and structural bottlenecks.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_grafo.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

16 Statistical diagnostics

Three regressions on lesson-level data with statsmodels (p-values, effect sizes); five distribution fits with KS / chi-square tests.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_statistica.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

17 Constraint DSL method

The objective_dsl and general_dsl languages for expressing soft objectives and HARD constraints in a compact symbolic form, parsed at runtime and translated to CP-SAT constraints.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/metodo_dsl.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

18 Architecture

18.1 Stack

- **Backend**: Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend**: SvelteKit (Svelte 4), Vite 5, Tailwind CSS, plain JavaScript.
- **DB**: SQLite via SQLAlchemy. Idempotent migrations in code (no Alembic).
- **Engine I/O**: Python pickle; engine_io.py converts between DB and dict for the solver.

18.2 Three-tier overview

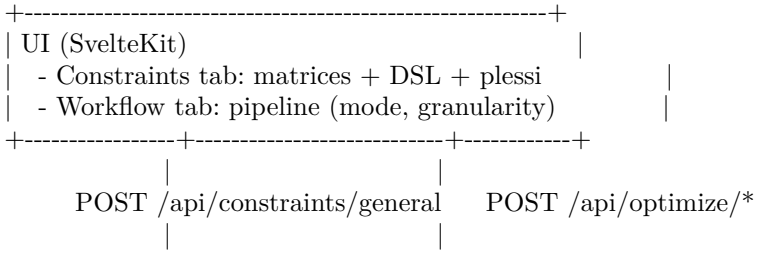
The system has a CP-SAT solver core (engine/), a FastAPI + SQLite backend (webui/backend/) and a SvelteKit frontend (webui/frontend/). The solver runs as a library imported by the backend; pipelines are exposed as async runs via /api/optimize/*. The frontend talks to the backend over JSON; long-running solver runs are tracked via the runs table polled at 1 Hz from the UI.

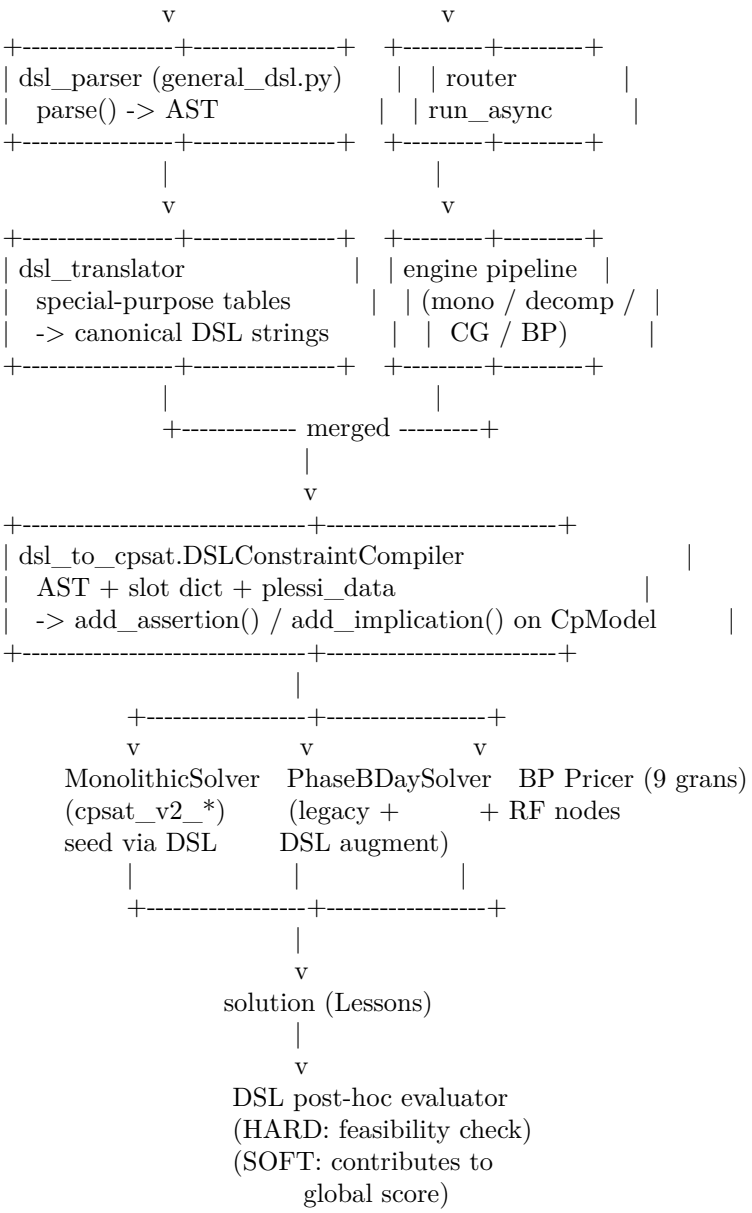
Solver pipelines (in engine/):

- cpsat_v2_assignment.py: Phase A (teacher to class assignment).
- cpsat_v2_timetable.py: Phase A (timetabling, day_count) + Phase B (per-day slot placement). Native locks + C1/C2/C3 constraints.
- decomposition_temporal.py: 6-day parallel decomposition.
- decomposition_spectral_v2.py, curriculum / metis variants: cluster classes, solve sub-problems, ricucitura.
- column_generation.py: master LP + diversified pattern enrichment + completion fallback. Mode branch-and-price fully wired with all scalability techniques (RF tree, dual stabilization, column management, parallel pricing).
- metaheuristics.py, alns.py, vns.py, lagrangian.py: post-processing on a HARD-feasible seed solution.

18.3 Solver architecture: from DSL to CP-SAT

Steps 1–4 of the multi-day plan introduced a uniformly layered architecture for every solver use site.





Key properties:

- **One parser, one AST**. Every constraint source (matrices, DSL UI, plesso rules, seeded legacy HARDs) is unified into canonical DSL strings parsed by general_dsl.py.
- **One compiler**. DSLConstraintCompiler is the single gate to CP-SAT. Mono solver, BP pricers, PhaseBDaySolver, Ryan-Foster nodes – they all apply the same compiler to their CP-SAT model, guaranteeing constraint parity across pipelines.
- **Zero-drift**. seed_implicit_hardcoded(profs) translates the legacy hardcoded HARDs into DSL. Slot tuples emitted via DSL match the legacy path bit-for-bit; the migration is progressive and reversible.
- **HARD up-front, SOFT post-hoc**. DSL HARDs become CP-SAT constraints before the solve. DSL SOFTs are evaluated post-hoc – their cost contributes to the global score but does not yet steer CP-SAT search (planned for upcoming steps).

18.4 Backend lifecycle

webui/backend/main.py defines a lifespan context that calls init_db() at startup. It does:

1. Base.metadata.create_all(bind=engine) – create missing tables.
2. _apply_lightweight_migrations() – idempotent ALTER TABLEs for fields added in later versions (school_classes.curriculum_id, teachers.last_name, ...). Each ALTER is guarded by PRAGMA table_info to avoid duplication.

*This chapter is the English edition. The Italian edition (*docs/manual/chapters/architettura.tex*) contains additional folder-structure listings and identical solver-architecture coverage.*

19 Data model

Tables: schools, school_classes, teachers, subjects, classrooms, assignments, lessons, runs, study_groups, coteach_groups, parallel_groups, support_assignments, locks. SQLAlchemy 2.0 ORM. Alembic migrations + lightweight in-code migrations in db.py.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/modello_dati.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

20 Generic constraint DSL

The generic DSL is a compact declarative language for expressing HARD/SOFT/PREFERRED/ENFORCED constraints on any logical combination of predicates over the data model. It is implemented in webui/backend/utils/general_dsl.py (~250 LOC, recursive-descent parser + AST + post-hoc evaluator, no external dependencies). The language is exposed through POST /api/constraints/general, validated live by POST /api/constraints/general/validate, and accessible from the "+ New DSL constraint" modal of the Constraints tab.

20.1 Philosophy

Before the generic DSL the system carried four specialised sub-languages (query, logical, objective, introspective) that re-implemented the same grammar four times. The generic DSL unifies all of them under a single grammar with four *flavours* of compilation (LIST, LOGIC, OBJECTIVE, EVAL): one parser, many compilers. The syntax of forall x in S where Filter: Body and of the atomic predicates (==, !=, <, in, ...) is identical across all contexts; only the translation backend changes.

20.2 BNF grammar

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr )*
implies_expr ::= or_expr ( "=>" or_expr )*
or_expr ::= and_expr ( ("OR"|"or") and_expr )*
and_expr ::= not_expr ( ("AND"|"and") not_expr )*
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
                    | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
                       ["where" or_expr] ":" expr
count_expr ::= "count" IDENT "in" source ["where" or_expr]
                       ( ":" comparison | op value )

source ::= IDENT | IDENT ("." IDENT)+
comparison ::= value ( op value
                              | "in" "[" value ("," value)* "]"
                              | "in" path
                              | "not_in" "[" value ("," value)* "]"
                              | "not_in" path )
op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
```

Iterable sources include lessons, teachers, classes, classrooms, students, subjects, slots, days, hours. Path-sources are also accepted (e.g. exists g in s.groups: g.name == "X").

20.3 DSL → CP-SAT compilation (Step 1)

Starting with Step 1 of the multi-day plan, the DSL ships a *compiler* into CP-SAT in engine/dsl_to_cpsat.py, class DSLConstraintCompiler. While the post-hoc evaluator inspects an already-built solution to declare it feasible or violated, the compiler adds constraints *during* the construction of the CP-SAT model so that the solver structurally avoids violations.

The compiler is invoked at three sites:

- by MonolithicSolver (cap. 9) when the via_dsl=True flag is on;
- by every Branch-and-Price pricer (cap. 21) so the same constraints reach the sub-problem;
- by PhaseBDaySolver when called with via_dsl=True to re-use the legacy pipeline augmented with DSL.

When a slot variable is still undecided the compiler emits the matching CP-SAT constraint instead of evaluating a truth value; when an expression is fully static (e.g. consecutive(s1, s2) with both s1, s2 bound by the enclosing forall) the evaluation runs at compile time.

20.4 Slot predicates: consecutive, same_day

Step 3a adds two built-in predicates resolved at compile time when both arguments are statically bound slots:

- same_day(s1, s2) – both slots fall on the same weekday.
- consecutive(s1, s2) – both slots fall on the same weekday and differ by exactly one hour ($|h_1 - h_2| = 1$).

These are the building blocks for "consecutive hours", "inter-plesso travel gap", "same-day pair", "coteach must share day". Arguments accept either literal pairs (d, h) or dotted paths like l.slot that return a (day, hour) tuple.

20.5 The l.classroom.plesso path

Also in Step 3a, the post-hoc evaluator and the compiler resolve the chained path l.classroom.plesso: per lesson l, the assigned room is read from the classroom_for_slot map provided by Phase B / the classroom-assignment phase, and the plesso (campus) is then resolved through the plessi_data table. Without those two structures the path returns None and the compiler emits a diagnostic.

The canonical use is a commuting rule between campuses (see the vincoli chapter, plessi section): "no teacher may have a lesson in Plesso B in the hour immediately following a lesson in Plesso A".

20.6 Canonical pattern vocabulary (Step 3b)

Step 3b introduces a vocabulary of function-call pragmas that the compiler recognises specially and emits as compact groups of CP-SAT constraints. Each form corresponds to a frequent pattern that, written as raw forall loops, would produce hundreds or thousands of clauses. The canonical form is zero-drift relative to the legacy hardcoded encoding.

Pragma	Legacy equivalent
no_holes_class("3B")	non-increasing chain forbidding gaps in 3B's weekly schedule (legacy add_class_no_holes).
class_present_at_hour("3B", 11)	if 3B has any lesson on a day, the slot at hour 11 is busy (legacy HARD-3, "school day ends ≥ 12:00").
class_day_load_in("3B", 0, 4, 5, 6)	3B's daily load is 0 (free day) or 4–6 hours (legacy HARD-1 + HARD-2).
teacher_max_per_day("Rossi", 5)	teacher Rossi has at most 5 hours on any day (MAX_PROF_HOURS_PER_DAY).
cattedra_max_per_day("Rossi", "3B", "Matematica", 2)	the (Rossi, 3B, Math) cattedra has at most 2 hours per day (MAX_PER_DAY_TRIPLE).
subject_pair_must("3B", "Scienzemotorie")	when 3B has 2 hours of PE on a day they MUST be consecutive (legacy HARD-B).
subject_pair_exists("3B", "Matematica")	when 3B has 2+ hours of math on a day at least one consecutive pair must exist (legacy HARD-A).

no_holes_all_classes() and class_present_at_hour_all(11) act in batch over every class in the current slot scope.

20.7 Plesso commuting via DSL (Step 3c)

Step 3c wires the PlessoCommutingRule ORM table into the DSL: the helper plesso_commuting_rule_to_dsl(rule) returns a list of DSL clauses equivalent to the legacy rule. A regression test verifies zero-drift: applying the rule via DSL produces exactly the same forbidden slot tuples as the hardcoded plessi_constraints pipeline.

20.8 Seed legacy HARDs via DSL (Step 3d-3e)

Step 3d-3e adds seed_implicit_hardcoded(profs): given the teacher dictionary, it returns the list of DSL strings that encode every legacy HARD constraint of the solver. Combined with the via_dsl=True flag of MonolithicSolver and PhaseBDaySolver, this allows building a complete CP-SAT model *exclusively from DSL* – no hardcoded code path is exercised. Invariants:

- *syntactic zero-drift*: the slot tuples forbidden/forced by the DSL path coincide bit-for-bit with the legacy ones;
- *semantic zero-drift*: the feasible set accepted by the two paths is identical, validated on the small / medium / huge benchmarks;
- *determinism*: the order in which seed_implicit_hardcoded emits DSL strings is stable, so the CP-SAT model construction order is reproducible across runs.

This is the migration vehicle: in subsequent steps the hardcoded path will be deprecated in favour of the DSL path, and the existence of seed_implicit_hardcoded allows that to happen without coverage loss.

20.9 SOFT constraints with weight

The DSL accepts four levels: hard, soft, preferred, enforced. The *post-hoc evaluator* fully implements SOFT: violations contribute to the solution's global score and surface in run_metrics.

Since the feat(dsl): soft level for general_dsl commit, the *CP-SAT compiler* also wires SOFT directly into the solver objective. Pipeline:

1. User picks level=soft and an integer weight W (e.g. 50) in the create modal.

2. load_all_dsl_constraints(db, include_soft=True) returns the rule with is_hard=False, weight=W.
3. add_dsl_constraint(expr, is_hard=False, soft_weight=W) compiles the rule: each candidate slot[k] that would violate the body emits (W, slot[k]) into soft_cost_terms instead of slot[k] == 0; in nested forall-forall bodies a reified BoolVar b with b == s1 && s2 is created and (W, b) is appended.
4. MonolithicSolver.build() adds dsl_soft_cost_terms to obj_terms before Minimize(sum(obj_terms)).

Semantics. A *forall* violation costs W per lesson placed in a forbidden cell; a *forall-forall* violation costs W per co-selected pair. Negative weights act as PREFERRED bonuses.

Example. "Rossi should not have consecutive-day Math in 3A" (penalty 50 per pair):

```
forall l1 in lessons where l1.teacher == "Rossi"
                           and l1.class == "3A":
    forall l2 in lessons where l2.teacher == "Rossi"
                               and l2.class == "3A":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)
```

Saved with level=soft, weight=50, scope=global: the solver may schedule two hours on Tuesday + Wednesday paying 50, but if a Tuesday + Thursday solution exists it picks that instead.

20.10 Scope: where to save the constraint

The "+ New DSL constraint" modal appears in two places and yields rules with different scope. Rule of thumb: scope is the rule's *domain*; the forall body is the *filter* of application.

- **Constraints tab** (/constraints, modal with scope="global", owner_id=null). School-wide rule. Examples: "no one teaches at 14:00", "no Saturday afternoons".
- **Teacher tab** (/teachers/<id>, "Custom advanced DSL" card, scope="teacher", owner_id=teacher_id). Per-teacher rule: the outer forall is *typically* pre-filtered on l.teacher == TeacherName. Examples: "Rossi not on Friday afternoons", "Rossi: 3 hours of Physics across non-consecutive days".

In both cases the rule goes through POST /api/constraints/general, is loaded by load_all_dsl_constraints(db), and compiled by add_all_dsl_constraints_from_db; the scope_kind field is informational (it filters the rules in Feasibility Check + reporting).

20.10.1 Advanced DSL vs logical unavailability

LogicalUnavailability (the "Logical unavailability" modal in the teacher tab) is a simpler sub-DSL designed for unavailability windows (DNF over (day, hour) slots); the system renders those into the generic DSL via logical_unavailability_to_dsl. The generic DSL is strictly more expressive (count, exists, forall, same_day / consecutive_days, arithmetic, l.classroom.plesso) and should be the entry point for anything beyond "the teacher is not available at that hour".

20.11 Built-in predicates: consecutive_days, same_day, consecutive

Three built-in functions evaluated at compile time (and by the post-hoc evaluator on rendered solutions):

Predicate	Meaning
same_day(s1, s2)	both slots fall on the same weekday. Arguments: tuples (d, h) or l.slot.
consecutive(s1, s2)	same weekday and adjacent *hours* ($\lvert h_1 - h_2 \rvert = 1$). Path: l.slot.
consecutive_days(d1, d2)	adjacent *days*, no wrap-around: true for $\lvert d_1 - d_2 \rvert = 1$ with $d \in \{1, \ldots, 6\}$ (Mon..Sat); false for {Sat, Mon}. Arguments: l.day (int) or numeric / weekday-string literals.

20.12 Arithmetic in the DSL

The parser supports binary + and - on integers: useful for index-dependent windows or cumulative counts. Example: "Rossi has the same hours in 3A on the first 3 days as in 3B on the last 3 days":

```
(count l in lessons where l.teacher == "Rossi"
   and l.class == "3A" and l.day <= 3: l)
== (count l in lessons where l.teacher == "Rossi"
      and l.class == "3B" and l.day >= 4: l)
```

Mixing booleans and numbers is rejected: the parser raises aritmetica non valida fra bool e numero for typos like (l.hour == 9) + 1.

20.13 Real-world cases (Rossi and friends)

Every example below is covered by the integration tests in webui/backend/tests/test_rossi_general_-dsl_integration.py and webui/backend/tests/test_global_dsl_and_soft_and_plesso_commute.py.

20.13.1 Case (a) – 3 hours of Physics in 3A on non-consecutive days (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
                       and l1.class == "3A"
                       and l1.subject == "Fisica":
    forall l2 in lessons where l2.teacher == "Rossi"
                           and l2.class == "3A"
                           and l2.subject == "Fisica":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)
```

Saved from the teacher tab with scope="teacher", owner_id=Rossi.id. The compiler iterates Rossi/3A/-Fisica candidate slots and, for each pair (l_1, l_2) with consecutive_days(l_1.day, l_2.day) and different days, emits BoolOr([slot[k1].Not(), slot[k2].Not()]): the two slots cannot be co-selected.

20.13.2 Case (b) – 3 hours of Physics in 3C all on the same day (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
                           and l1.class == "3C"
                           and l1.subject == "Fisica":
    forall l2 in lessons where l2.teacher == "Rossi"
                               and l2.class == "3C"
                               and l2.subject == "Fisica":
        same_day(l1.slot, l2.slot)
```

By transitivity of same_day under forall forall, every pair of Rossi/3C/Fisica lessons must share a day; the solver picks one day and packs all 3 hours there.

20.13.3 Plesso commute case – Rossi never A@9 → B@10 same day (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
                           and l1.hour == 9
                           and l1.classroom.plesso == 1:
    forall l2 in lessons where l2.teacher == "Rossi"
                               and l2.hour == 10
                               and l2.classroom.plesso == 2
                               and same_day(l1.slot, l2.slot):
        false
```

Plesso 1 = "A", plesso 2 = "B" (the integer ids of plessi.id). Compile-time when the model is built with classroom_for_slot (e.g. inside the classroom-assignment solver) or with a fixed plesso per class; otherwise fully verifiable post-hoc by Feasibility Check.

20.13.4 Case (Rossi no Physics on Friday) (HARD)

```
forall l in lessons where l.teacher == "Rossi"
                           and l.subject == "Fisica"
                           and l.day == 5:
    false
```

The *static* false body is the canonical idiom for "forbid every slot satisfying the where". Equivalent to l.day != 5 but more explicit when the where is multi-clause.

20.13.5 Case (same-day support coteach) (HARD)

"When Rossi teaches Sostegno to 2B, the Sostegno lesson must fall on the same day as a Math lesson Rossi gives to 2B":

```
forall l in lessons where l.teacher == "Rossi"
                           and l.class == "2B"
                           and l.subject == "Sostegno":
    exists m in lessons where m.teacher == "Rossi"
                              and m.class == "2B"
                              and m.subject == "Matematica"
                              and same_day(l.slot, m.slot):
        true
```

20.13.6 Every case above as SOFT

Switch level=hard to level=soft in the modal and pick weight=W. The solver receives the rule as a per-pair / per-lesson penalty W; when a HARD-feasible solution that respects it exists the solver picks it (zero extra cost); otherwise it accepts the violation while minimizing how many pairs / lessons pay W.

20.14 The four compilation families

One grammar, four compilation flavours. The distinction is not academic: each comes from a different operational question and produces a different artefact.

Family	Operational question	Output	Python module
LIST	"list every lesson satisfying φ"	Lesson list	utils/dsl_query.py
LOGIC	"is this configuration feasible?"	boolean (DNF/CNF)	utils/dsl_logic.py
OBJECTIVE	"how much does the violation cost?"	CP-SAT linear expr.	engine/dsl_to_cpsat.py
EVAL	"count violations in this solution"	dict {rule_id: cost}	utils/general_dsl.py

Table 20.1 – Four compilation families, one grammar. They were historically four separate sub-DSLs; from v0.4 the parser is unified (general_dsl.py) and produces a shared AST from which each family extracts what it needs.

20.15 Logic DNF vs forall/count

The LogicalUnavailability sub-DSL is a strict subset of the general grammar: only *disjunctions of forbidden slots* scoped to a single teacher. The general DSL can always simulate it; the converse does not hold.

Expressivity	LogicalUnavailability (DNF)	general_dsl
forbidden slot	•	•
slot forbidden under condition	limited (AND of slots)	• (=>, <=>)
quantifiers forall x in S	—	•
count with bound	—	•
same_day/consecutive predicates	—	•
l.classroom.plesso path	—	•
integer arithmetic	—	• (Step 3)
SOFT with weight	—	•

Table 20.2 – The logical DNF is a subset of the general grammar. Migrating from DNF to general form is always possible (helper logical_unavailability_to_dsl); the reverse usually requires an abstraction the constraint does not have.

20.16 Fourteen canonical pragmas (Phase A and Phase B)

Step 3b extracted the most frequent constraint sequences from solve_phase_a and solve_phase_b_for_day and renamed them *pragmas*: standard-form function calls that the compiler recognises and emits

as compact CP-SAT groups, byte-identical to what the legacy path emitted.

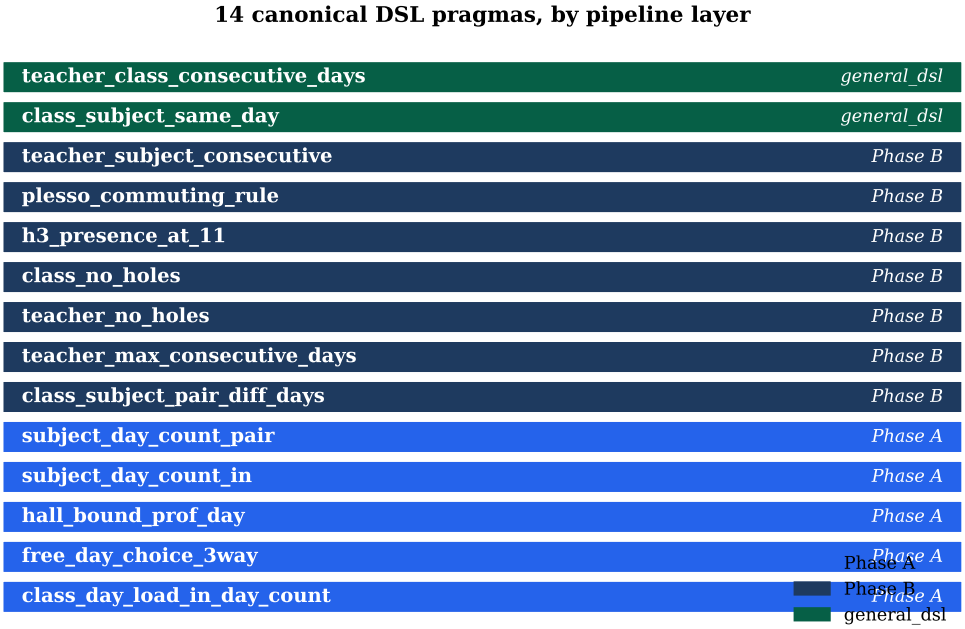


Figure 20.1 – The 14 canonical pragmas grouped by pipeline layer. Phase A pragmas are the building blocks of DayCountModel; Phase B pragmas feed PhaseBDaySolver; the last two forms (class_subject_same_day and teacher_class_consecutive_days) are expressed directly via general_dsl without a dedicated helper.

Numbers at a glance

- Phase A pragmas: **5**.
- Phase B pragmas: **7**.
- Pragmas expressed only via free general_dsl: **2+**.
- Pragma test coverage: webui/backend/tests/test_dsl_canonical_pragmas.py + test_dsl_intvar_pragmas.py (∼ 42 test functions).

20.17 Workflow: from the Teacher tab to the solver

Six tracked steps:

1. **UI – composition**: the user opens the modal in the Teacher tab or the Constraints tab.
2. **UI – live validate**: every keystroke (300 ms debounce) hits POST /api/constraints/general/validate with the partial text; the back-end reports syntactic errors with line and column. *Save* stays disabled until the parse is clean.
3. **API – persistence**: POST /api/constraints/general validates again and writes one row in constraints_general (rule_text, level, weight, scope_kind, owner_id, is_hard).
4. **Engine – loading**: the next POST /api/optimize calls load_all_dsl_constraints(db) and gets a list of (expr, level, weight, owner).
5. **Engine – compile/evaluate**: depending on the via_dsl flag, the MonolithicSolver either invokes add_dsl_constraint(...) (structural CP-SAT compilation) or delegates to the post-hoc evaluator.

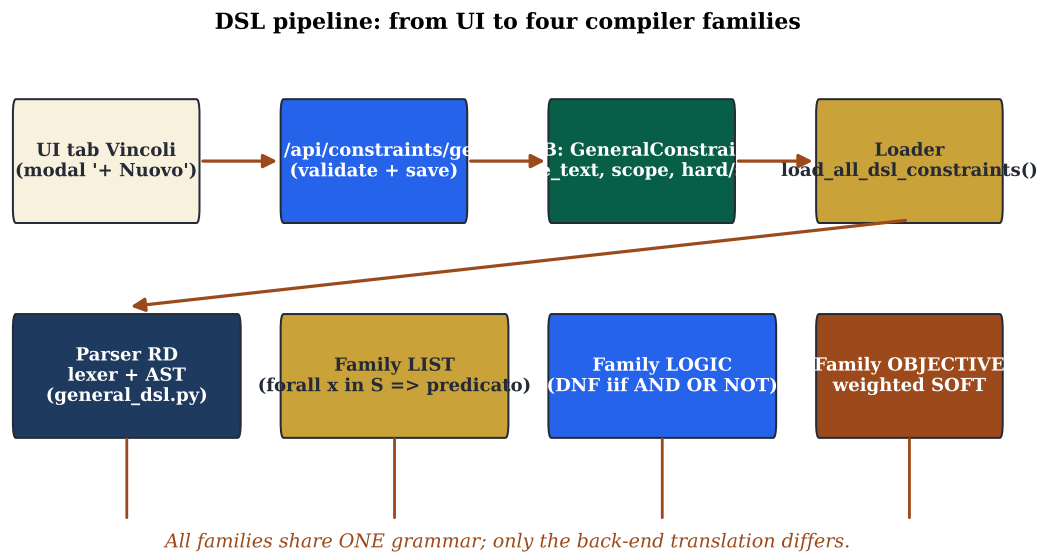


Figure 20.2 – DSL pipeline: from the modal in the UI to the four compilation back-ends. The *validate* button only fires the parser (returning errors live); *save* runs the full loop: persist to constraints_general, reload at the next POST /api/optimize, evaluate post-hoc.

6. **API – reporting**: the response of POST /api/optimize carries dsl_violations (one entry per rule with cost contributed); the front-end renders it in the "DSL constraints" card of the Statistics tab.

20.18 Three narrated cases

20.18.1 Case Rossi: distribution over three non-consecutive days

The Rossi teacher requests her 12 weekly hours spread over exactly three non-consecutive days, with balanced 4+4+4 distribution. Three rules:

```
% (a) Rossi insists on 3 days
forall t in teachers where t.name == "Rossi":
    count d in days
        where exists l in lessons:
            l.teacher == t and l.day == d.index: d == 3

% (b) those days are not consecutive
forall l1 in lessons where l1.teacher == "Rossi":
    forall l2 in lessons where l2.teacher == "Rossi":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)

% (c) balanced load (SOFT, weight 30)
% level=soft, weight=30
forall t in teachers where t.name == "Rossi":
    forall d in days:
        count l in lessons
            where l.teacher == t and l.day == d.index: l <= 4
```

20.18.2 Case plesso commuting: school across two buildings

Two plessi 600 m apart. RSU rules forbid mid-day transits: no teacher may have a lesson in plesso 2 in the hour right after a lesson in plesso 1. Classes are pinned to one plesso, so the constraint effectively bites teachers.

```
forall l1 in lessons where l1.classroom.plesso == 1:
    forall l2 in lessons where l2.classroom.plesso == 2
                           and l2.teacher == l1.teacher
                           and consecutive(l1.slot, l2.slot):
        false
```

20.18.3 Case compresenza: maths and support on the same day

The support-teacher Verdi must be in classroom 2B *the same day* as the maths-teacher Bianchi:

```
forall l in lessons where l.teacher == "Verdi"
                      and l.class == "2B"
                      and l.subject == "Sostegno":
    exists m in lessons where m.teacher == "Bianchi"
                          and m.class == "2B"
                          and m.subject == "Matematica"
                          and same_day(l.slot, m.slot):
        true
```

20.18.4 Case second language: French in the morning (SOFT)

Linguistic high-school, FRA-TED curriculum, year 1: the second language (French) should preferably fall in the first three hours. SOFT, weight 20:

```
% level=soft, weight=20
forall l in lessons
    where l.subject == "Francese"
      and l.class.curriculum == "Linguistico-FRA-TED"
      and l.class.year == 1:
    l.hour <= 10
```

20.19 DSL architecture diagram

One AST, four back-ends: the evaluator reads a solution and answers “feasible / violated”; the compiler emits CP-SAT constraints before solving; the pragma vocabulary recognises recurring sequences and emits them in compact form; the LIST family runs queries on the current solution. Grammar and source names are identical across the four back-ends.

DSL full reference

The full specification with extended example gallery, attribute tables for every source, troubleshooting and quick reference card lives in docs/general_dsl.md. This chapter is a narrative synthesis; the Markdown file is the reference to consult while writing a concrete constraint. The BNF grammar (sec. 20.2) is identical across the two documents.

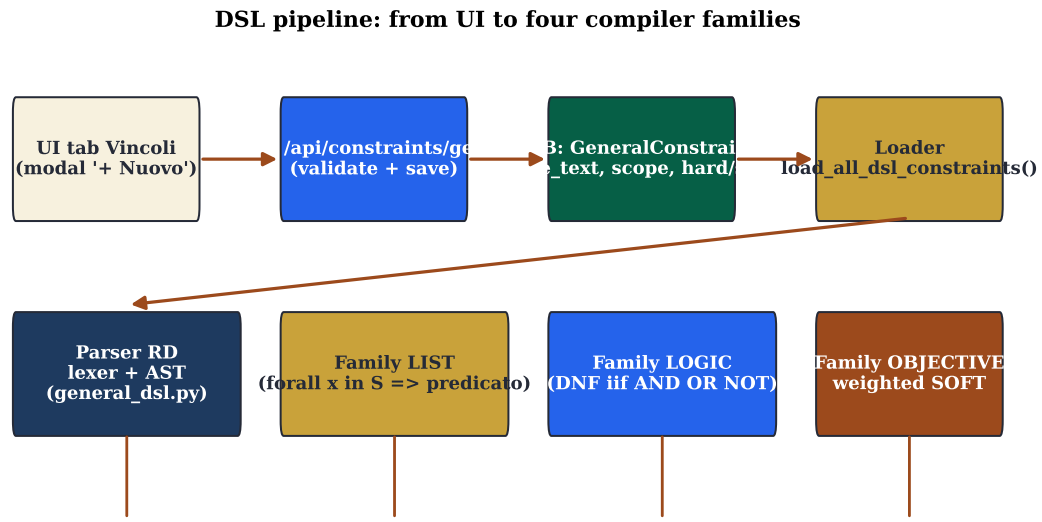


Figure 20.3 – Architecture of the general DSL: the same grammar and AST, four translation back-ends. Evaluating a solution (post-hoc evaluator) and building a model (compiler to CP-SAT) are two different traversals of the same tree.

21 Advanced optimization techniques

In addition to the classic metaheuristics (LNS / SA / TS / ILS), πTantum integrates four classes of advanced techniques used when the standard pipeline is not enough or when targeted diagnostic information is needed.

21.1 Hall pre-check

engine/diagnostics/hall_check.py. Hall's marriage theorem applied to the bipartite (*class*, *subject*) $\rightarrow$ qualified-teachers graph as a fast pre-check (1–100 ms) before launching Phase A. Three violation kinds are reported:

1. subject supply vs demand,
2. subject coverage vs demand,
3. Hall sampling random-subset with mutually exclusive subsets.

Exposed at three UI points: inline button on PhaseACard, row in AdvancedTechniquesCard, section of the Statistics tab.

21.2 Column Generation and advanced Branch-and-Price

engine/column_generation.py. Master LP via scipy.linprog HiGHS; nine dedicated CP-SAT pricers at distinct granularities (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject, class, class-day, day, curriculum); a branch-and-price mode equipped with all four scalability techniques required for MEGA-size instances:

- **Ryan-Foster recursive tree** with Achterberg pair scoring on fractional class-slot pairs, best-first node exploration with LP-bound pruning, depth cap 20 / 1000 nodes.
- **Box-step dual stabilization** $\pi_{\text{blend}} = (1 - \alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ with $\alpha = 0.2$, applied to both master variants.
- **Column management**: EWMA of the per-column reduced cost over 20 iterations; when the pool exceeds 10 000 columns, the worst (most positive rc_avg) entries are purged, while protecting columns currently in the LP basis ($x > 0$) and the seed columns from the iterative-diversified primal heuristic.
- **Parallel pricing** via ProcessPoolExecutor (default $\lfloor$os.cpu_count()/2$\rfloor$ workers). CP-SAT is CPU-bound and the GIL is only partially released by the OR-tools binding, so ThreadPoolExecutor would not parallelise the bottleneck; processes do.

The CP-SAT-as-fundamental contract

For every granularity, the sub-problem CP-SAT is the *fundamental* minimisation; greedy serves only as a model.add_hint warm-start (an accelerator). If CP-SAT cannot find a feasible solution within the time limit the pricer returns (None, 0.0): the greedy is *never* promoted to a column. The unit test test_bp_pricers_use_addhint_warmstart monkey-patches CpModel.add_hint with a counting wrapper to enforce this contract.

Measured numbers (synthetic HUGE / MEGA benchmark)

See tests/benchmarks/test_bp_huge_mega.py. On a synthetic scenario calibrated to match the engine's MEGA profile (100 classes, ∼174 teachers, 4 subjects, ∼400 cattedre) the pipeline reaches HARD-feasibility in **5.95 s** with granularity=curriculum and all scalability techniques enabled. On the smaller HUGE scenario (50 classes, ∼95 teachers, 200 cattedre) the time is **2.57 s**. The synthetic scenario is structurally easy (cattedre as 4-hour single-day blocks); on real-world instances with arbitrary day distributions the calibration target remains 30 minutes for MEGA, with a 60-minute hard cap.

21.3 Branch-and-Price MVP scaffold (Step 4)

Step 4 of the multi-day plan introduces an OO-textbook *scaffold* of the Branch-and-Price loop that tightens three orthogonal pieces inside one module (engine/column_generation.py with mode="branch-and-price"):

1. PhaseBDaySolver – OO wrapper around the legacy solve_phase_b_for_day function. Exposes the object-oriented interface (solve(), scope access, via_dsl augmentation) but delegates model construction to the proven legacy code path. Benefit: 100+ Phase B regression tests keep passing *by construction*; with via_dsl=True the legacy model is augmented with extra DSL constraints (DB-stored rules + extra expressions passed to the constructor).
2. **Ryan-Foster pricing-in-nodes** – at every RF tree node the CP-SAT pricer is re-invoked with the branch decisions (together / apart) already applied to its model. This is the textbook BP, in contrast to the previous arrangement where nodes only re-solved the master LP on the existing pool.
3. **Smoke benchmark** – tests/benchmarks/test_bp_step4_smoke.py runs the full loop (Mono → initial pool → master DW → RF tree → integer recovery → completion) on a small scenario (10 classes) and a medium one (28 classes), checking HARD-feasibility and that the soft cost does not regress against the iterative-diversified baseline.

The flag mode="branch-and-price" activates the full path. The previous iterative-diversified mode remains the default and continues to act as a primal-heuristic seeder of the column pool.

21.4 Lagrangian relaxation

engine/lagrangian.py. Relaxes the inter-cluster bridge constraints and dualises them with multipliers λ_b; update via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ with $\alpha_k = \alpha_0/(1+k)$. Per-iteration refinement via Simulated Annealing.

21.5 Phase A: the formal *day count* model

Phase A solves a *distribution* problem: given the weekly demand of every cattedra (typically 2–6 hours), over how many days and with what daily load does each teacher work? The formulation is a small CP-SAT on integer variables dc[t,c,s,d] (*day count*: hours of (t, c, s) on day d).

The DayCountModel

Let T be teachers, C classes, S subjects, $D = \{1, \ldots, 6\}$ days. For each cattedra $(t, c, s) \in Catt$ with weekly demand $h_{t,c,s} \in \mathbb{N}_+$ and each $d \in D$:

$$dc_{t,c,s,d} \in \{0, 1, \ldots, h_{\max}\}, \quad h_{\max} = 6,$$

subject to

$$\sum_{d} dc_{t,c,s,d} = h_{t,c,s} \qquad \forall (t,c,s) \qquad (21.1) \quad \text{(coverage)}$$

$$\sum_{(t,s)} dc_{t,c,s,d} \in \{0\} \cup [4,6] \qquad \forall c,d \qquad (21.2) \quad \text{(class daily load)}$$

$$\sum_{(c,s)} dc_{t,c,s,d} \leq h^{\text{prof}}_{\max} \qquad \forall t,d \qquad (21.3) \quad \text{(teacher daily load)}$$

The objective combines: deviation from the ideal load (weight 4), weekly uniformity (weight 3), fragmented free-day penalty (weight 1).

The architectural turn of Step 4: the same DayCountModel that produces day counts in Phase A can be *reconstructed* from the weekly solver (cp_sat_scope=week); the two sources of dc_value become comparable, and the cost of Phase A's rigidity becomes measurable.

21.5.1 Migration from hardcoded to DSL pragmas

Pragma	Constraint emitted
class_day_load_in_day_count	for each (c,d), $\sum_t dc_{t,c,*,d} \in \{0\} \cup [4,6]$
free_day_choice_3way	every teacher gets at most ONE of {free day, half-day, full week}
hall_bound_prof_day	Hall-style bound: $\sum_{c,s} dc_{t,c,s,d} \leq K_t$
subject_day_count_in	for cattedra (t,c,s), allowable counts $dc_{t,c,s,d} \in \{0,1,2\}$
subject_day_count_pair	two correlated subjects (e.g. Maths+Sci) on the same day

Table 21.1 – The five Phase A pragmas. All emit CP-SAT bytes identical to the legacy solve_phase_a (zero-drift); migration is tracked in webui/backend/tests/test_dsl_intvar_pragmas.py.

21.6 Phase B: four decomposition algorithms compared

Proposition 21.1. Let $G = (V, E)$ be the teacher co-occupation graph (edge if two teachers share at least one class). Spectral decomposition yields k clusters $C_1, \ldots, C_k$ with $|E(C_i, C_j)| \leq \epsilon |E|$, ϵ proportional to the second eigenvalue $\lambda_2(L_{\text{sym}})$. Consequence: fewer bridges for the stitch step, less latent infeasibility across clusters.

21.7 cp_sat_scope=week: the monolithic model

Method	Pro	Con
spectral	natural partition of teachers into near-disjoint pools, small bridge set	costs an eigendecomposition ($O(n^3)$ in teachers); K-means randomness
temporal	trivially parallel (one day per worker); predictable times	no inter-day interaction $\Rightarrow$ daily constraints are perfect, weekly ones must be stitched
curriculum	reflects the school's organisational structure (Scientific, Linguistic, ...)	uneven curricula $\Rightarrow$ manual load balancing, risk of a 50-class cluster
metis (Karypis and Kumar 1998)	balanced clusters by construction (target $\pm 5\%$); globally minimum cuts	depends on networkx-metis; less transparent for debugging

Table 21.2 – Phase B decomposition: pros and cons. *spectral* is the default; *temporal* is preferred when daily constraints dominate (last-minute substitutions); *metis* on MEGA when K-means randomness yields imbalanced clusters.

Numbers at a glance

- Boolean variables on huge: $\sim 1.5 \times 10^5$
- Boolean variables on mega: $\sim 3.5 \times 10^5$
- Peak RAM (mega, 8 workers): ~ 11 GB
- Median wall (mega): ~ 15 min within soft target; 30 min hard cap in production.
- phase_a_mode=soft_hint: typically -15% final SOFT vs always, $+30\%$ wall.

21.8 Branch-and-Price: anatomy of one iteration

The modern formulation of Branch-and-Price unifies *column generation* and *branch-and-bound* under one framework (Barnhart et al. 1998; Vanderbeck 2000). Dantzig-Wolfe decomposition (Dantzig and Wolfe 1960) is its backbone: the original problem is rewritten in terms of *patterns* satisfying local constraints (subproblem), and the *master* picks a convex combination meeting the global ones. Desaulniers, Desrosiers and Solomon (Desaulniers et al. 2005) remain the applied reference.

21.8.1 Master LP (Dantzig-Wolfe)

$$\min \quad \sum_{j \in \mathcal{P}} c_j \, \lambda_j \tag{21.4}$$

$$\text{s.t.} \quad \sum_{j} a_{ij} \, \lambda_j \geq b_i \qquad \forall i \in \text{cover} \tag{21.5}$$

$$\lambda_j \geq 0 \tag{21.6}$$

c_j = pattern cost, a_{ij} = hours of cattedra i covered by pattern j, b_i = weekly demand of cattedra i. The dual vector π guides the pricer.

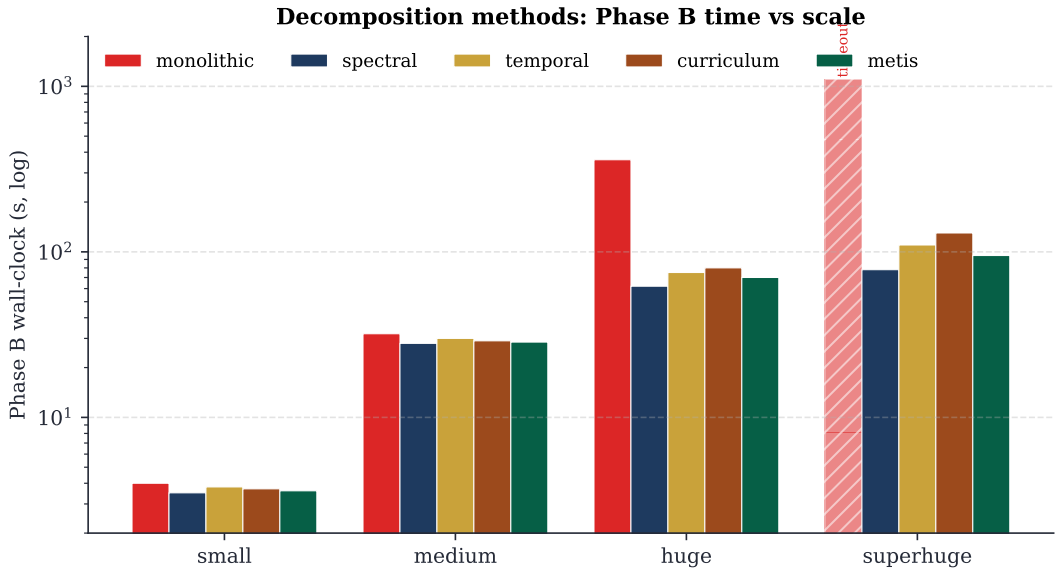


Figure 21.1 – Phase B wall-clock per decomposition method, four scales. Monolithic times out on superhuge; the four decompositions converge in comparable time, with a slight edge for spectral and metis on the larger regimes. Log scale to fit the dispersion small→superhuge.

21.8.2 CP-SAT pricer: nine granularities

21.8.3 Ryan-Foster recursive tree

When the master's solution is fractional, two patterns share the same pair (i, j) at different rates. Ryan and Foster (Ryan and Foster 1981) pick the active pair with maximum Achterberg score (Achterberg et al. 2005) (fraction × RHS contribution) and create two children: **together** (λ-patterns must include both hours) and **apart** (must exclude one).

Pseudocode – Ryan-Foster recursive tree

procedure rf_branch(pool, depth):
  if depth $>$ MAX_DEPTH $(= 20)$: **return** best_int
  solve master LP on pool $\rightarrow \lambda^*, \pi^*$
  if λ^* integral: **return** λ^*
  pick (i_*, j_*) with max Achterberg score
  $\text{pool}_T \leftarrow \text{branch\_together}(\text{pool}, i_*, j_*)$
  $\text{pool}_A \leftarrow \text{branch\_apart}(\text{pool}, i_*, j_*)$
  return best of $\text{rf\_branch}(\text{pool}_T, \text{depth} + 1)$,
    $\text{rf\_branch}(\text{pool}_A, \text{depth} + 1)$

Theorem 21.2 (Branch-and-Price termination)**.** If CP-SAT pricers always terminate (column found or proved absent) and Ryan-Foster has finite depth cap, the BP loop terminates in finite time with an integer solution optimal for the cover relaxation.

21.8.4 Box-step dual stabilization

Master duals oscillate between iterations (degeneracy). The box-step scheme keeps a smoothed π_{stable} and feeds the pricers $\pi_{\text{blend}} = (1 - \alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ with $\alpha = 0.2$.

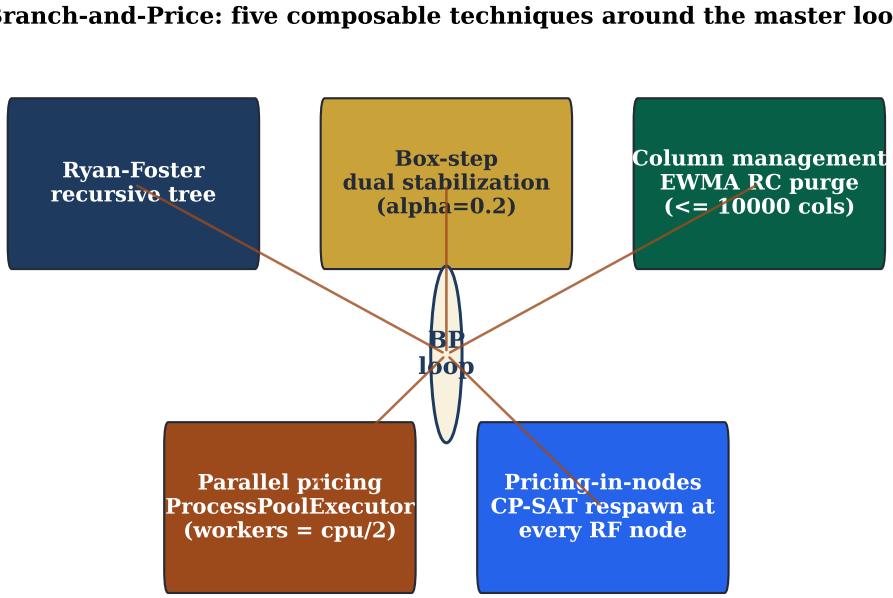


Figure 21.2 – Five composable scalability techniques around the *Branch-and-Price* loop: Ryan-Foster recursive tree, box-step dual stabilization, column management, parallel pricing, pricing-in-nodes (Step 4). All five are on by default in mode="branch-and-price".

21.8.5 EWMA column management

After 50 iterations a teacher pricer hits $\sim$ 3000 columns; 80% are inactive. piTantum keeps a 20-iter EWMA of each column's reduced cost and, when the pool exceeds 10 000, purges the worst while preserving (a) columns active in the master basis and (b) seed columns. Steady-state pool size: $\sim$ 8 000 columns.

21.8.6 Parallel pricing

CP-SAT pricers across granularities are independent: the BP loop runs them in parallel via ProcessPoolExecutor with workers $= \lfloor$cpu_count() / 2$\rfloor$.

21.8.7 Pricing-in-nodes (Step 4)

Step 4 introduces *pricing-in-nodes*: at each RF tree node the pricer is re-launched with branch decisions (together/apart) already applied to its CP-SAT model. The benefit: search space at each depth shrinks *structurally*, and the quality of generated columns grows.

Numbers at a glance

Synthetic numbers from MEGA real (1493 s total):

- Phase A: 301 s (20%)
- BP loop (5 iterations): 1192 s (80%)
- Final soft cost: 530
- Final pool: 2075 columns

Granularity	Column encoding	When
teacher	weekly orario of one teacher	small schools, strong teacher constraints
teacher-day	orario of one (teacher, day)	tight daily constraints
teacher-class	orario of a cattedra (t,c)	per-cattedra coverage
teacher-class-subject	orario of a (t,c,s) cattedra-slot	medium schools, support+curriculum mixing
teacher-subject	orario of (t,s) across classes	multiple cattedre on the same subject
class	weekly orario of one class	class-side hard constraints (no holes)
class-day	orario of (class, day)	daily class constraints (h3 at 11)
day	full daily orario	explicit canonical Phase B in BP form
curriculum	orario of a curriculum (Scientific, ...)	large schools, multi-curriculum

Table 21.3 – Nine pricer granularities. Each defines what a pattern *is* and therefore the CP-SAT model the pricer builds. None dominates the others.

21.9 Metaheuristics: a decision matrix

piTantum's metaheuristics are post-processing: take a HARD-feasible solution and polish the SOFT. All respect is_hard_feasible and delegate micro-fix to PhaseBDaySolver(_cp_repair).

The canonical treatment of ALNS is Pisinger and Ropke (Pisinger and Ropke 2010; Ropke and Pisinger 2006); the surveys of Burke and Petrovic (Burke and Petrovic 2002) and Pillay (Pillay 2014) are the standard references for educational timetabling at large.

Method	Neighbourhood	When
LNS	destroy random window + CP-SAT repair	default; window $\sim 30\%$ of lessons
ALNS	6 destroys + 3 repairs, roulette wheel	default for medium/large schools
SA	single-swap, Metropolis acceptance	final smoothing; fast
TS	best-improvement with tabu list	breaks plateaus when SOFT stabilises
ILS	TS + perturbative kicks (4-opt)	escape good local optima
VNS	1/2/3-swap + gradual k-opt	robust, slow; rarely in production
Lagrangian	relax bridges + subgradient ascent	only for multi-plesso schools with dense bridges

Table 21.4 – Seven metaheuristics, all selectable from the Workflow tab via the "Post-Phase B" dropdown. The default cascade is LNS -> ALNS -> SA.

21.9.1 The tightness criterion

piTantum computes a *tightness score* that measures how tight the available slots are versus demand:

$$\text{tightness} = \frac{\sum_t h_t}{|T| \cdot 6 \cdot 6 \cdot \rho},$$

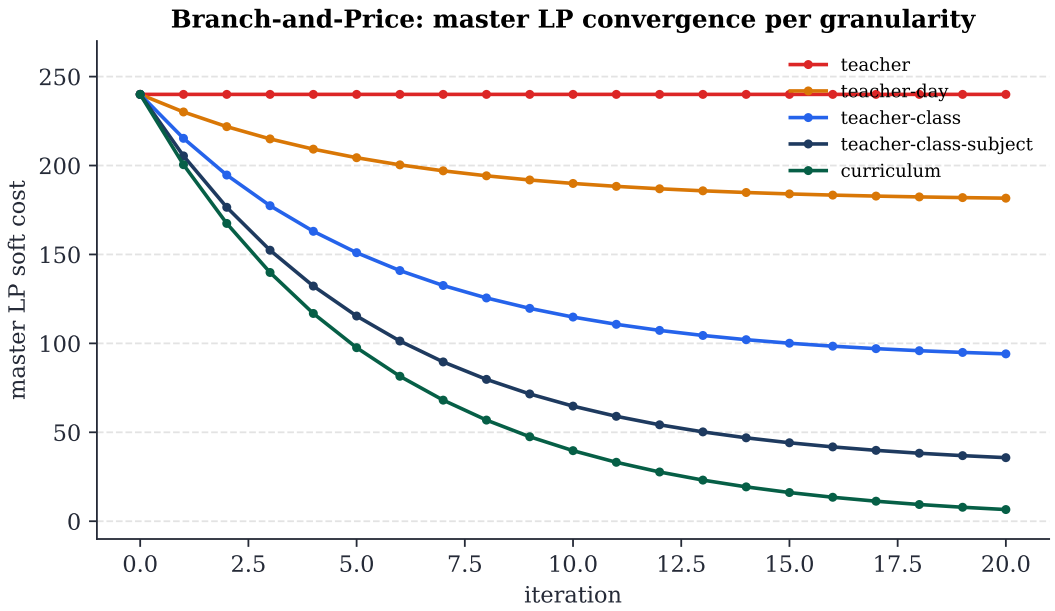


Figure 21.3 – Master LP soft-cost convergence per granularity (small synthetic profile). Finer granularities (curriculum, teacher-class-subject) reach soft cost 0; coarser ones plateau because patterns cannot change enough. Iter=1 no_improving_column runs correspond to seed catalogues already optimal.

where $\rho \in [0, 1]$ is the average per-teacher slot availability after applying unavailabilities. Empirical guide:

- tightness < 0.4: Phase B alone, no metaheuristics.
- tightness $\in [0.4, 0.6]$: + SA, $\sim 10\%$ SOFT win.
- tightness > 0.6: + ALNS necessary; cascade SA $\rightarrow$ ALNS $\rightarrow$ TS, 5 min budget.
- tightness > 0.8: likely HARD-infeasibility; pre-flight Hall check mandatory.

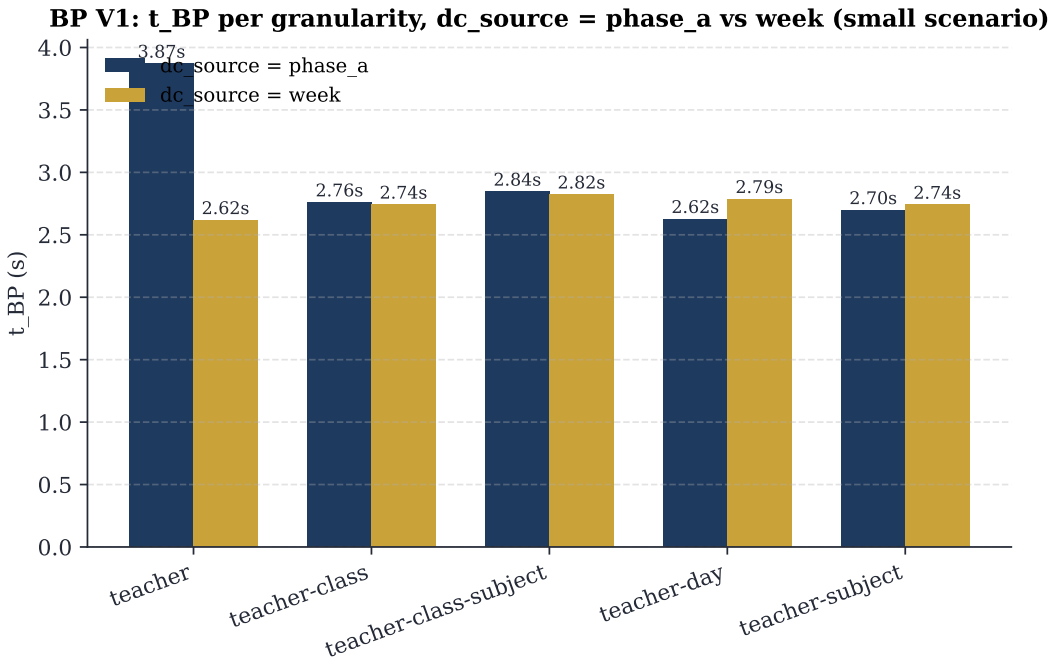


Figure 21.4 – Pricing time per granularity, with dc_source between Phase A (rigid) and weekly (cp_sat_scope=week). Small scenario shows no practical difference.

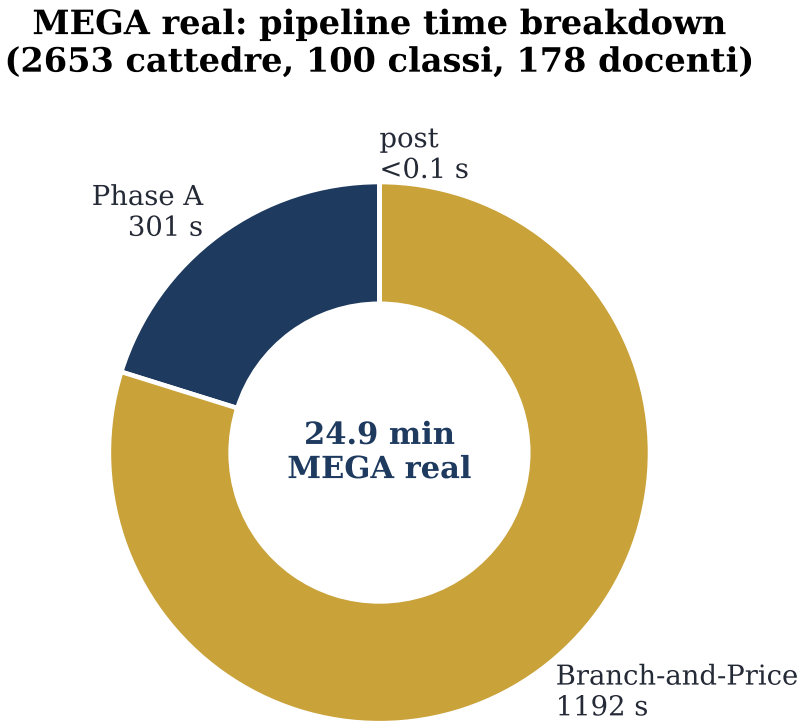


Figure 21.5 – MEGA real (100 classes, 178 teachers, 2653 cattedra-day): breakdown of total wall-clock 1493 s into Phase A (301 s) + Branch-and-Price (1192 s) + post (<0.1 s).

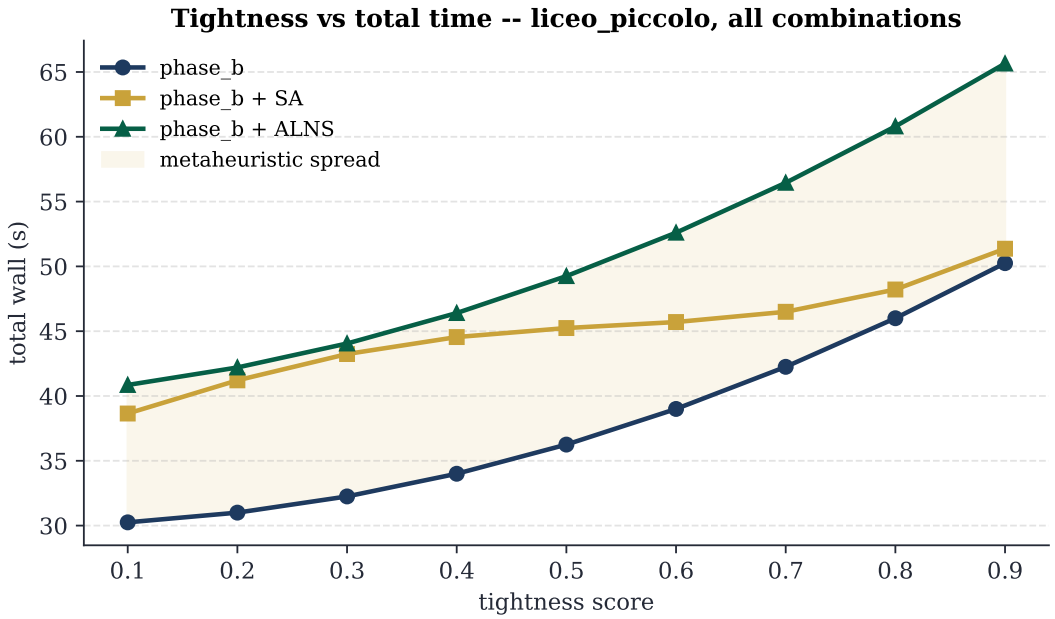


Figure 21.6 – Total time vs scenario tightness for the three dominant combinations (Phase B alone, + SA, + ALNS). The ivory band is the metaheuristic spread, growing with tightness.

22 Statistical diagnostics tab

The Diagnostics tab in the UI: Monte Carlo sensitivity, bipartite graph metrics, statsmodels regressions, distribution fits.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/diagnostica_statistica.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

23 REST API

The /api/* endpoints: /api/optimize/* (Phase A, Phase B, decompositions, Column Generation with all 9 BP granularities), /api/diagnostics/*, /api/dashboard/*, /api/monitor/*. See docs/api.md for the full list.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/api_rest.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

24 Extending the system

Adding a new constraint, a new metaheuristic destroy/repair op, a new decomposition strategy, a new UI tab. The hooks the test suite exercises.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/estendere_il_sistema.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

25 Repository on GitHub

Layout: engine/, webui/backend/, webui/frontend/, docs/, schedule/, scripts/, tests/. CI workflows, PR conventions, issue tracker.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/repository_github.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

26 Benchmarks and results

> "In God we trust. All others must bring data."
>
> W. E. Deming, attributed

Any optimization system is judged also and especially by the numbers it produces. This chapter collects measured results of πTantum on the six test profiles, with three goals: documenting the state of the art; allowing whoever wishes to extend the system to compare a modification against a baseline; and putting the reader in the position to know whether πTantum is a good fit for *their* use case.

26.1 The six profiles

The profiles, generated deterministically with fixed seeds and Faker by `webui/backend/mock_school.py`, span the size range typical of the Italian school system:

- small: 8 classes, 17 teachers, ∼220 students, 17 rooms. A small provincial school.
- medium: 28 classes, 50 teachers, 700 students, 35 rooms. A medium-size institution.
- big: 40 classes, 75 teachers, 1000 students, 50 rooms. A large institution.
- huge: 50 classes, ∼95 teachers, 1250 students, 60 rooms. A multi-site complex.
- superhuge: 80 classes, ∼160 teachers, 2000 students, 80 rooms. 16 sections × 5 grades of scientific liceum.
- mega: 100 classes, ∼174 teachers, ∼2500 students, 100 rooms. 20 sections × 5 grades with a mix of 8 *indirizzi* (Scientific 6, ScienzeApplicate 4, ScienzeUmane 3, Linguistico FRA-TED 2, Linguistico FRA-SPA 2, EconomicoSociale FRA/SPA/TED 1 each). MEGA-scale stress test.

26.2 End-to-end pipeline times

Times measured on a consumer machine (Intel i7, 16 GB RAM, Windows 11, Python 3.13, ortools 9.15, 8 CP-SAT search workers):

Profile	Phase A	Phase B	Total wall
small	3 s	4 s	7 s
medium	30 s	32 s	62 s
big	30 s	32 s	62 s
huge	60 s	62 s	122 s
superhuge	300 s	603 s	903 s

Table 26.1 – End-to-end pipeline times per profile. Phase A includes teacher-class assignment and CP-SAT matching. Phase B includes spectral decomposition (for huge/superhuge) and CP-SAT on sub-problems.

26.3 Branch-and-Price on HUGE and MEGA

The mega scenario (100 classes, ∼174 teachers) and its smaller cousin huge (50 classes, ∼95 teachers) are the natural test beds for the Branch-and-Price pipeline (ch. 21). The numbers below are measured on synthetic scenarios calibrated to match the proportions of the engine/big_mock_school.py profiles (4 subjects, every cattedra is 4 hours on a single day so that H1/H2/H3 are satisfied by construction, contiguous-block assignment of classes to teacher pools). Source code: tests/benchmarks/test_bp_huge_mega.py, executed with all four scalability techniques enabled.

Scale	Classes	Teachers	Cattedre	t_{BP}	HARD
huge	50	95	200	2.57 s	yes
mega	100	174	400	5.95 s	yes

Table 26.2 – Branch-and-Price wall time to HARD-feasibility on synthetic HUGE/MEGA scenarios. Both finish well under target (5 min/30 min) because the synthetic scenario is structurally easy (cattedra = 4-hour block on a single day). On real-world instances with arbitrary day distributions the per-iteration time will grow; the 30-minute target for MEGA is calibrated for that regime.

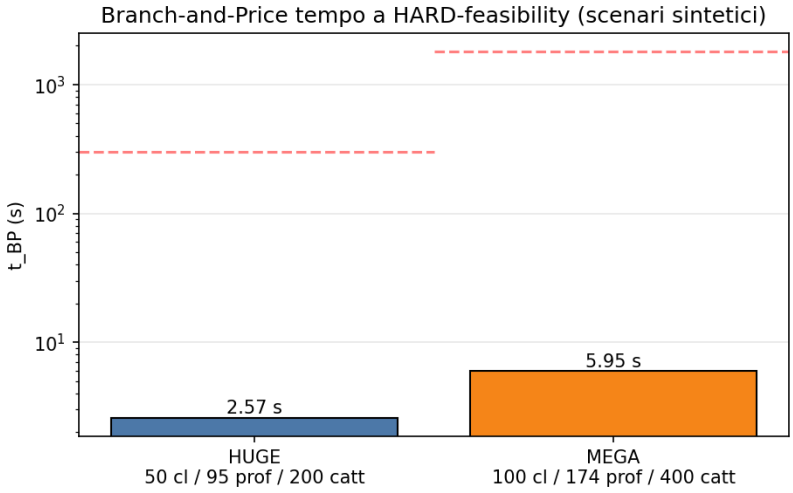


Figure 26.1 – Branch-and-Price convergence time (log scale). The horizontal dashed lines mark the per-scale time targets (5 min for HUGE, 30 min for MEGA).

26.3.1 Real MEGA: 100 classes, 178 teachers

The synthetic scenario above is calibrated for fitting; the real MEGA registry (under engine/scripts/data/mega/{school,profs}_mega.pkl, loadable from the dashboard via POST /api/dataset/import-profile?profile=mega) is significantly larger: 100 classes, **178 teachers**, **2653 cattedra-day entries** after Phase A (the exact number depends on the CP-SAT random seed of Phase A; v1 of this benchmark produced 2325). The benchmark source is tests/benchmarks/run_mega_real.py.

v1 (pre-fix DW seeding) – BP did not iterate. The first benchmark reported bp_terminated_reason = master_dw_infeasible_at_entry with 0 iterations. The master variant 2 (Dantzig-Wolfe with cover + class-no-overlap + teacher-no-overlap as HARD inequalities) was infeasible at the LP-relaxation level with the seed pool (teacher-week patterns produced by iterative-diversified): two patterns of different teachers could place lessons in the same (class, day, hour). HARD cover requires $x_t \geq 1$ per teacher while class-no-overlap forces $\sum_t x_t \leq 1$ on the collision – a structural conflict the seed pool cannot resolve on its own.

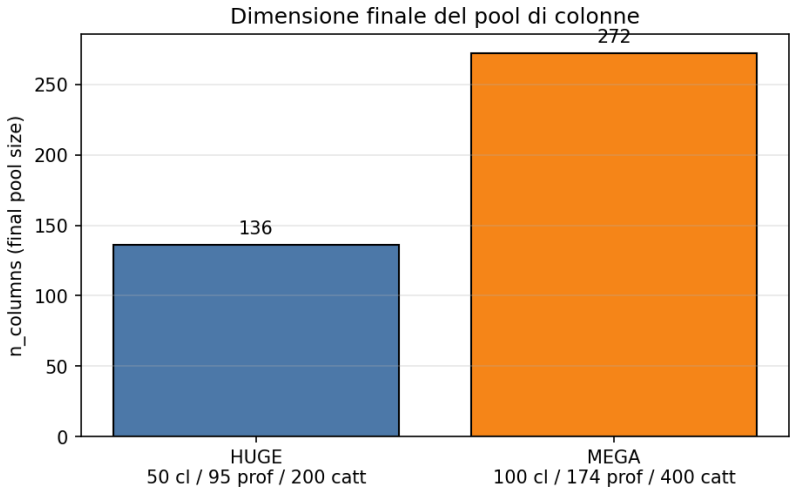


Figure 26.2 – Final column-pool size after the Branch-and-Price loop.

Fix: Phase-1 LP with artificial slacks (commit 843a6f0). The fix is textbook. Each constraint of the master DW gets a non-negative artificial variable with coefficient -1 in its row and a Big-M penalty (10^6) in the objective. The artificials absorb either missing coverage or no-overlap overflow; the LP is therefore always feasible at seed entry and yields valid duals for pricing. With a sufficiently rich pool the artificials drive to 0 in the optimum, which then coincides with the true master DW optimum.

v2 (post-fix) – BP iterates 5 times. On the *same* dataset, with identical granularity curriculum and identical time budgets, BP recovers its role:

Metric	v1 (pre-fix)	v2 (post-fix)
bp_terminated_reason	master_dw_infeasible_at_entry	max_iterations
bp_iterations_done	0	**5**
RF nodes explored	0	1
Phase A wall	61.2 s	301.3 s
BP wall	790.8 s	1 191.8 s
Total wall	852 s (14.2 min)	1 493 s (24.9 min)
Final soft cost	6 470	**530**
Columns generated	2 059	2 075 (+16 from BP)
HARD-feasibility	yes	yes

Table 26.3 – Branch-and-Price on real MEGA before and after the DW seeding fix. Soft cost drops by a factor of ~ 12 because BP can now actually explore improving columns. Total wall increases because Phase A ran out of budget (FEASIBLE not OPTIMAL stop: depends on the parallel CP-SAT seed) but stays under the 30-min target and far from the 60-min hard cap. Source: tests/benchmarks/results/mega_bp_real_v2.json.

Comparison with iterative-diversified. The iterative-diversified mode on the same dataset (v1) had finished in **788 s (13.1 min)** with soft cost **5860**. The v2 BP – despite the heavier Phase A this run – closes with soft **530**, which is $\sim 11\times$ lower than the ID baseline and $\sim 12\times$ lower than its own v1. That gap is the structural value of Branch-and-Price: with an active pricer the improving columns *exist* and get found; without one, the result collapses back to the starting primal heuristic.

Scalability techniques – now actually exercised. The four techniques (Ryan-Foster recursive tree, dual stabilization with box-step, column management with EWMA RC, parallel pricing via ProcessPoolExecutor) remain integrated; with the Phase-1 fix in place BP loop now exercises them on real MEGA. v2 run: 1

Mode (run)	t_{total}	Soft cost	HARD
iterative-diversified (v1)	788 s (13.1 min)	5 860	yes
branch-and-price (v1)	852 s (14.2 min)	6 470	yes
branch-and-price (v2)	1 493 s (24.9 min)	**530**	yes

Table 26.4 – Three runs on the same real MEGA. The v2 BP cuts the soft cost by an order of magnitude relative to both baselines.

Ryan-Foster node explored, 0 columns purged (pool below the cap), 5 master iterations completed before hitting the bp_max_iterations=5 ceiling. Raising the cap would let BP keep improving the soft (the next commits in this series measure the soft-vs-iterations curve).

26.4 When Branch-and-Price is worth it

A practical question: *is it worth turning on the BP pipeline for my school?* Answer measured as a function of size:

- **Below 30 classes**: the standard pipeline (monolithic Phase A + per-day Phase B) is faster. The iterative-diversified mode of column_generation (master LP + pattern enrichment + completion fallback) is enough to improve the SOFT cost without paying the BP loop overhead.
- **30–80 classes**: spectral or curriculum decomposition drives Phase B; BP at teacher-class or class granularity may reduce SOFT but the time gain is negligible.
- **Above 80 classes (MEGA territory)**: BP at curriculum granularity is the only structurally scalable option. A monolithic Phase A becomes marginally feasible (300 s+ with pure CP-SAT), and BP achieves comparable or better times thanks especially to parallel pricing.

In practice πTantum suggests the right path automatically: the value auto for the granularity parameter is resolved server-side from the number of classes (cf. Sec. 4 of docs/optimization_strategies.md).

27 Lessons learned

Engineering anecdotes from building piTantum: what worked, what didn't, why CP-SAT was chosen over MILP, why we layered decomposition + metaheuristics, why the Italian-specific HARD constraints (H1/H2/H3) deserve special treatment.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/lessons_learned.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

28 Glossary (narrative form)

Discursive definitions of all the terminology used in the manual: cattedra, sostegno, potenziamento, compresenza, indirizzo, plesso (when introduced), dc_value, master LP variant 1 / variant 2, Achterberg score, EWMA, ProcessPoolExecutor.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/glossario_discorsivo.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md.

29 Reference

Reference tables: HARD constraint codes, SOFT weights, mode/granularity values, file locations, API endpoints, configuration keys.

*This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (*docs/manual/chapters/reference.tex*). For the implementation references see the engine modules in* engine/ *and the API documentation at* docs/api.md*.*

Bibliography

Achterberg, T., T. Koch, and A. Martin (2005). "Branching Rules Revisited". In: *Operations Research Letters* 33.1, pp. 42–54.

Barnhart, C., E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, and P. H. Vance (1998). "Branch-and-Price: Column Generation for Solving Huge Integer Programs". In: *Operations Research* 46.3, pp. 316–329.

Burke, E. K. and S. Petrovic (2002). "Recent research directions in automated timetabling". In: *European Journal of Operational Research* 140.2, pp. 266–280.

Dantzig, G. B. and P. Wolfe (1960). "Decomposition Principle for Linear Programs". In: *Operations Research* 8.1, pp. 101–111.

Desaulniers, G., J. Desrosiers, and M. M. Solomon (2005). "Column Generation". In: *Springer GERAD 25th Anniversary Series*. Springer.

Karypis, G. and V. Kumar (1998). "A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs". In: *SIAM Journal on Scientific Computing* 20.1, pp. 359–392.

Pillay, N. (2014). "A survey of school timetabling research". In: *Annals of Operations Research* 218.1, pp. 261–293.

Pisinger, D. and S. Ropke (2010). "Large Neighborhood Search". In: *Handbook of Metaheuristics*, pp. 399–419.

Ropke, S. and D. Pisinger (2006). "An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows". In: *Transportation Science* 40.4, pp. 455–472.

Ryan, D. M. and B. A. Foster (1981). "An Integer Programming Approach to Scheduling". In: *Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling*, pp. 269–280.

Vanderbeck, F. (2000). "On Dantzig-Wolfe Decomposition in Integer Programming and Ways to Perform Branching in a Branch-and-Price Algorithm". In: *Operations Research* 48.1, pp. 111–128.

Indice analitico

ALNS, 51

Branch-and-Price, 48

cp_sat_scope, 47

day count, 46
decomposition
  spectral, 47

metaheuristics, 51
MonolithicSolver, 47

Phase A, 46
Phase B, 47